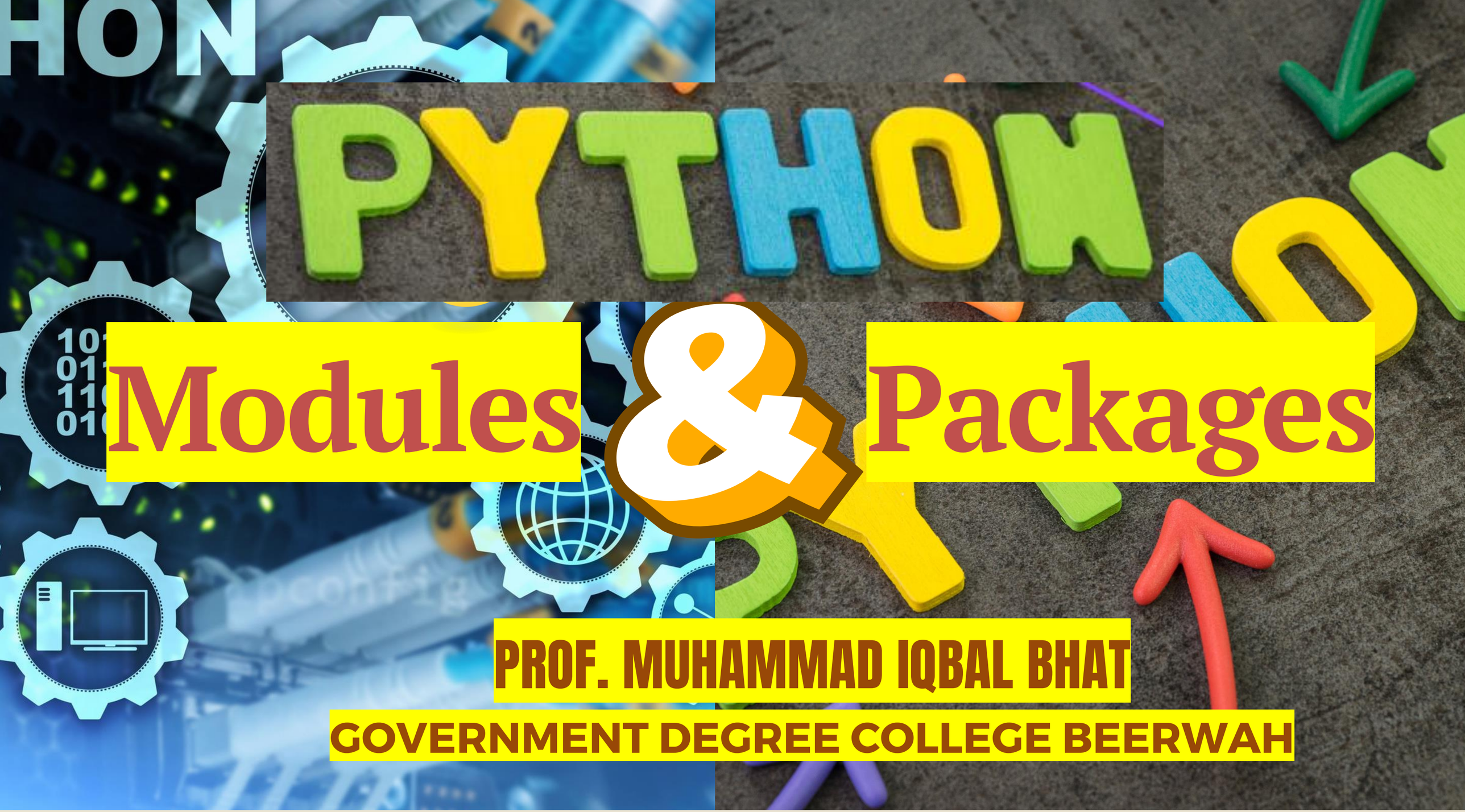


Modules & Packages

PROF. MUHAMMAD IQBAL BHAT

GOVERNMENT DEGREE COLLEGE BEERWAH

Modular Programming

Modular programming refers to the process of breaking a large, unwieldy programming task into separate, smaller, more manageable subtasks or modules. Individual modules can then be cobbled together like building blocks to create a larger application.

Why Modularity?

- **Simplicity**: Rather than focusing on the entire problem at hand, a module typically focuses on one relatively small portion of the problem. If you're working on a single module, you'll have a smaller problem domain to wrap your head around. This makes development easier and less error-prone.
- **Maintainability**: Modules are typically designed so that they enforce logical boundaries between different problem domains. If modules are written in a way that minimizes interdependency, there is decreased likelihood that modifications to a single module will have an impact on other parts of the program. (You may even be able to make changes to a module without having any knowledge of the application outside that module.) This makes it more viable for a team of many programmers to work collaboratively on a large application.

Why Modularity?

3. **Reusability**: Functionality defined in a single module can be easily reused (through an appropriately defined interface) by other parts of the application. This eliminates the need to duplicate code.
4. **Scoping**: Modules typically define a separate namespace, which helps avoid collisions between identifiers in different areas of a program.

Functions, modules and packages are all constructs in Python that promote code modularization.

What are modules?

- A module is a file containing Python definitions and statements. It can define functions, classes, and variables, and can be imported to other files.
- Modules allow us to organize your code into separate, reusable files, making it easier to manage and maintain your projects.

Using inbuilt modules?

- Import Module: To import a module, use the import statement. For example, to import the math module, you would use the following statement:

```
import math
```

- Access Module Content: Once a module is imported, you can access the functions and variables it contains using dot notation. For example, to access the sqrt() function from the math module, you would use the following statement:

```
print(math.sqrt(16))
```

The import Statement:

Module contents are made available to the caller with the import statement. The import statement takes many different forms, shown below

- import <module_name> : The statement import <module_name> only places <module_name> in the caller's symbol table. The objects that are defined in the module remain in the module's private symbol table.

From the caller, objects in the module are only accessible when prefixed with <module_name> via dot notation

module_name.object_name

Several comma-separated modules may be specified in a single import statement:

```
import <module_name>[, <module_name> ...]
```

The import Statement:

2. from <module_name> import <name(s)> An alternate form of the import statement allows individual objects from the module to be imported directly into the caller's symbol table

```
from <module_name> import <name(s)>
```

Following execution of the above statement, <name(s)> can be referenced in the caller's environment without the <module_name> prefix

Because this form of import places the object names directly into the caller's symbol table, any objects that already exist with the same name will be overwritten

The import Statement:

3. from <module_name> import * It is even possible to indiscriminately import everything from a module at one

```
from <module_name> import *
```

This will place the names of all objects from <module_name> into the local symbol table, with the exception of any that begin with the underscore (_) character

This isn't necessarily recommended in large-scale production code. It's a bit dangerous because you are entering names into the local symbol table en masse.

The import Statement:

4. from <module_name> import <name> as <alt_name> It is also possible to import individual objects but enter them into the local symbol table with alternate names

```
from <module_name> import <name> as <alt_name>[, <name> as <alt_name> …]
```

This makes it possible to place names directly into the local symbol table but avoid conflicts with previously existing names

The import Statement:

5. import <module_name> as <alt_name> You can also import an entire module under an alternate name

```
import <module_name> as <alt_name>
```

This makes it possible to place names directly into the local symbol table but avoid conflicts with previously existing names

The import Statement:

6. import within function Definition Module contents can be imported from within a function definition. In that case, the import does not occur until the function is called

```
def mib(num):
    import math
    print(math.sqrt(num))
mib(16)
```

Executed at 2023.10.17 09:00:52 in 75ms

```
4.0
```

However, Python 3 does not allow the indiscriminate import * syntax from within a function

Creating and Using Modules:

- Create a Module: To create a module, you can save a Python file with a .py extension. For example, let's create a module named my_module.py:

```
# mymod.py.py
def greet(name):
    return f"Hello, {name}!"
```

- 2. Using the Module: You can use the functions and variables defined in a module in another Python script by importing it:

```
# main.py
import mymod
message = mymod.greet("Alice")
print(message)
```

The Module Search Path

- What happens when Python executes the statement:

import mymod

When the interpreter executes the above import statement, it searches for mymod.py in a list of directories assembled from the following sources:

- The directory from which the input script was run or the current directory if the interpreter is being run interactively
- The list of directories contained in the PYTHONPATH environment variable, if it is set. (The format for PYTHONPATH is OS-dependent but should mimic the PATH environment variable.)
- An installation-dependent list of directories configured at the time Python is installed

The Module Search Path

The resulting search path is accessible in the Python variable sys.path, which is obtained from a module named sys:

```
import sys
print(sys.path)
```

Executed at 2023.10.17 08:23:22 in 64ms

```
 \\venv\\lib\\site-packages', 'C:\\Users\\Prof MIB
 Creations\\PycharmProjects\\6thsemesterpython\\venv\\lib\\site-packages
 \\win32', 'C:\\Users\\Prof MIB
 Creations\\PycharmProjects\\6thsemesterpython\\venv\\lib\\site-packages
 \\win32\\lib', 'C:\\Users\\Prof MIB
 Creations\\PycharmProjects\\6thsemesterpython\\venv\\lib\\site-packages
 \\Pythonwin']
```

The Module Search Path

Thus, to ensure your module is found, you need to do one of the following::

- Put mymod.py in the directory where the input script is located or the current directory, if interactive
- Modify the PYTHONPATH environment variable to contain the directory where mod.py is located before starting the interpreter
- Or: Put mymod.py in one of the directories already contained in the PYTHONPATH variable
- Put mymod.py in one of the installation-dependent directories, which you may or may not have write-access to, depending on the OS

The Module Search Path

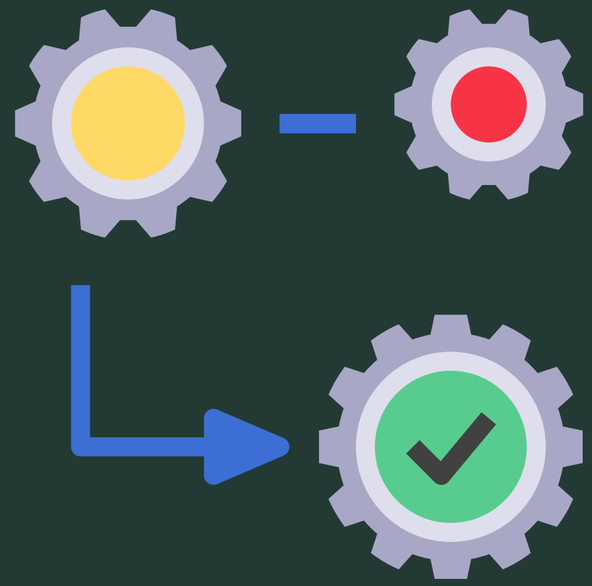
Modify sys.path:

There is actually one additional option: you can put the module file in any directory of your choice and then modify sys.path at run-time so that it contains that directory. For example, in this case, you could put mymod.py in directory C:\Users\mib and then issue the following statements

```
sys.path.append(r'C:\Users\mib')
import  mymod
```

Once a module has been imported, you can determine the location where it was found with the module's __file__ attribute:

```
mymod.__file_
```



Executing a Module as a Script

Executing Module as Script

Any .py file that contains a module is essentially also a Python script, and there isn't any reason it can't be executed like one

```
\BCA 3rd Skill> python mymodule.py
```

There are no errors, so it apparently worked. Granted, it's not very interesting. As it is written, it only defines objects. It doesn't do anything with them, and it doesn't generate any output.

If we modify the above Python module so it does generate some output when run as a script

```
>>> import mymodule.py
This is a message string in module
```

Unfortunately, now it also generates output when imported as a module

__main__

How to Distinguish between file loaded as module and run as standalone script

When a .py file is imported as a module, Python sets the special dunder variable __name__ to the name of the module. However, if a file is run as a standalone script, __name__ is (creatively) set to the string '__main__'. Using this fact, you can discern which is the case at run-time and alter behavior accordingly

```
if (__name__ == '__main__'):
    print('Executing as standalone script')
    print(s)
    print(a)
    foo('quux')
    x = Foo()
    print(x)
```

How Reload a Module?

If you make a change to a module and need to reload it, you need to either restart the interpreter or use a function called reload() from module importlib:

```
import mibmodule
```

Executed at 2023.10.17 11:31:12 in 94ms

```
mibmodule.greeting("Iqbal")
```

Executed at 2023.10.17 11:31:26 in 169ms

```
 Hello Iqbal
```

```
import importlib
```

Executed at 2023.10.18 08:18:24 in 15ms

```
importlib.reload(mibmodule)
```

Executed at 2023.10.18 08:18:39 in 122ms

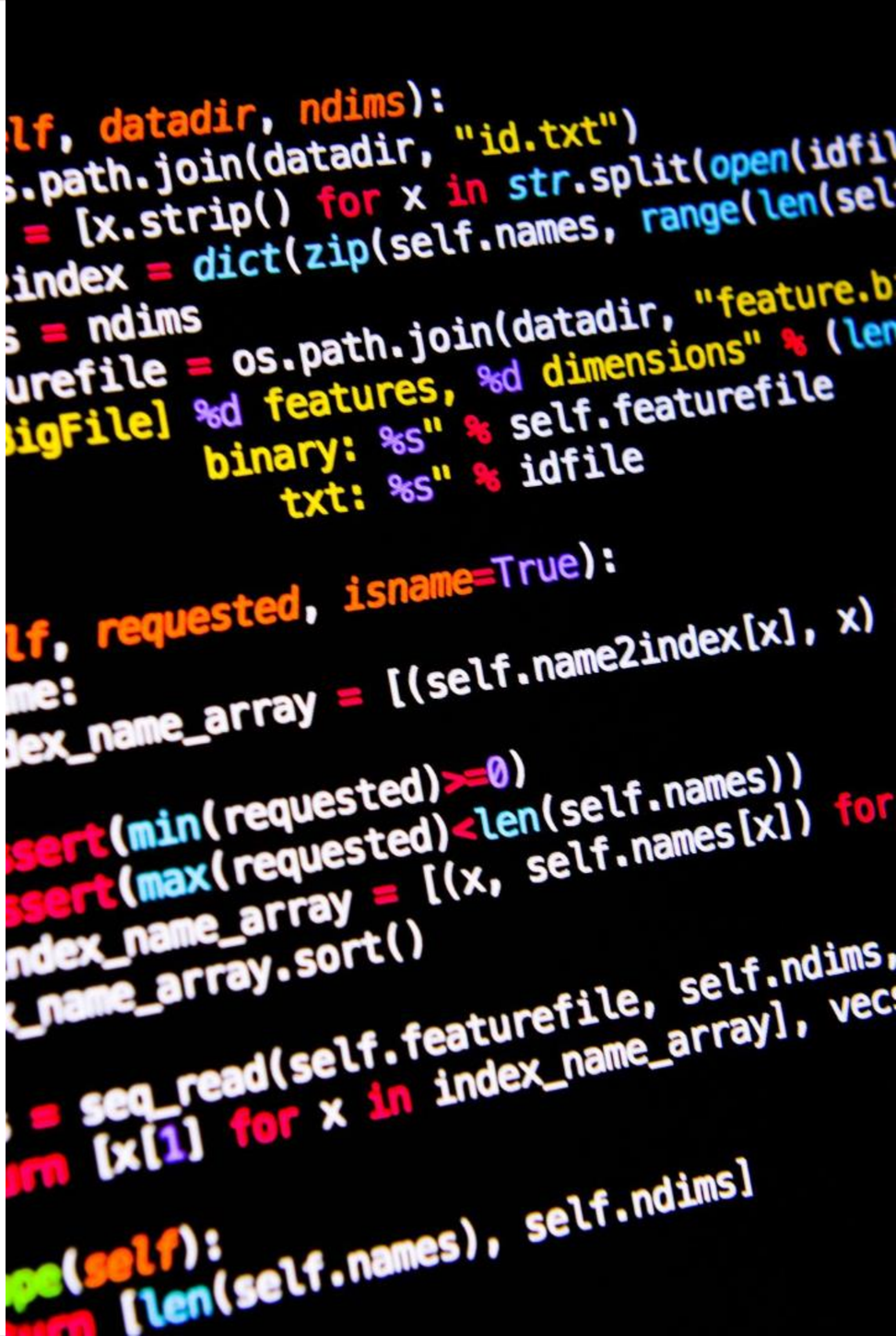
```
 <module 'mibmodule' from 'C:\\Users\\Prof MIB
 Creations\\PycharmProjects\\6thsemesterpython\\BCA 3rd Skill\\mibmodule
 .py'>
```

Python Packages

Python Packages: Creation, Importing, and Usage

Welcome to our comprehensive guide on Python packages. In this presentation, we will explore everything you need to know about creating, importing, and using Python packages.

by Prof. Muhammad Iqbal Bhat



An Introduction to Python Packages

1 **What Are Python Packages?**

Suppose you have developed a very large application that includes many modules. As the number of modules grows, it becomes difficult to keep track of them all if they are dumped into one location. This is particularly so if they have similar names or functionality. You might wish for a means of grouping and organizing them.

2

Packages allow for a hierarchical structuring of the module namespace using dot notation. In the same way that modules help avoid collisions between global variable names, packages help avoid collisions between module names.

Python Packages

1

Hierarchical Structure

Use dot notation to create hierarchical structuring of the module namespace.

2

Avoid Name Collisions

Packages help avoid collisions between module names.

3

Example

Directory structure with packages and modules.

Creating and Using a Python Package

Package Structure and Module files

Creating a Package: To create a package, you need to create a directory (folder) and place your module files within it. The directory can contain an __init__.py file to be recognized as a package

```
my_package/
├── __init__.py
├── module1.py
├── module2.py
```

Using a Package

You can use modules from a package in your Python scripts by importing them

```python
# main.py
from my_package import module1, module2

result1 = module1.add(3, 4)
result2 = module2.multiply(5, 6)

print(result1)
print(result2)
```

Importing a Python Package

1

Using the import Statement

```
import pkg
```

```
import pkg.mod1, pkg.mod2
```

2

Importing Specific Modules from a Package

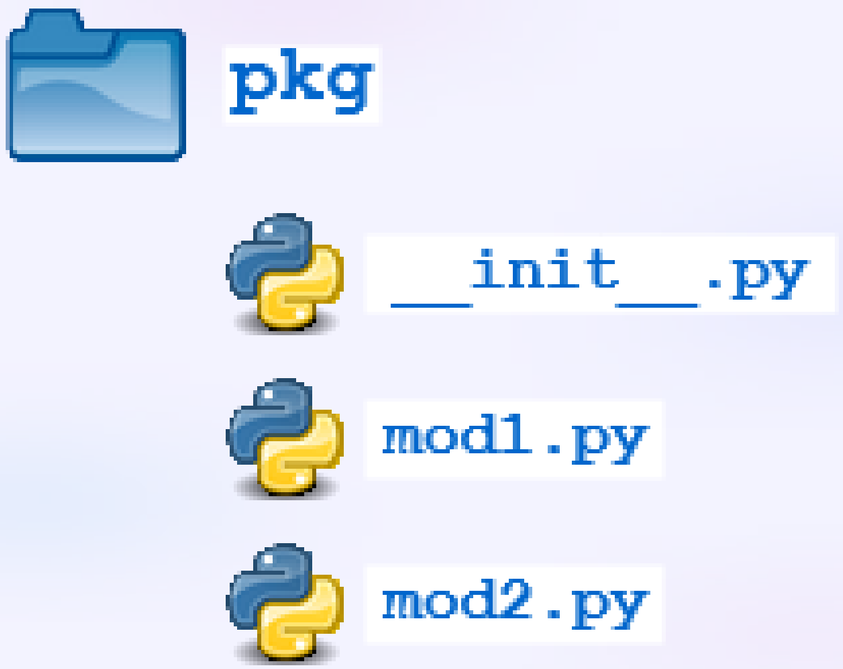
```
from <package_name> import <modules_name>[, <module_name> ...]
from <package_name> import <module_name> as <alt_name>
```

```
from <module_name> import <name(s)>
```

3

Aspects of Namespace

```
from <module_name> import <name> as <alt_name>
```



Package Initialization

Use the `__init__.py` file to execute package initialization code and define package-level data.

1 Package Initialization

The `__init__.py` file is invoked when the package or a module in the package is imported.

2 Initialization Code

You can define and execute package initialization code in the `__init__.py` file.

__init__.py

```python
print(f'Invoking __init__.py for {__name__}')
A = ['quux', 'corge', 'grault']
```

3 Package-Level Data

Initialization code can be used to initialize package-level data that can be accessed by modules in the package.

Importing * From a Package

1 — Default Behavior `from <package_name> import *`

`import *` statement for a package does not import anything by default.

2 — The all List

Define the `__all__` list in `__init__.py` to import specific modules when using `import *`.

pkg/__init__.py

Python

```python
__all__ = [
        'mod1',
        'mod2',
        'mod3',
        'mod4'
        ]
```

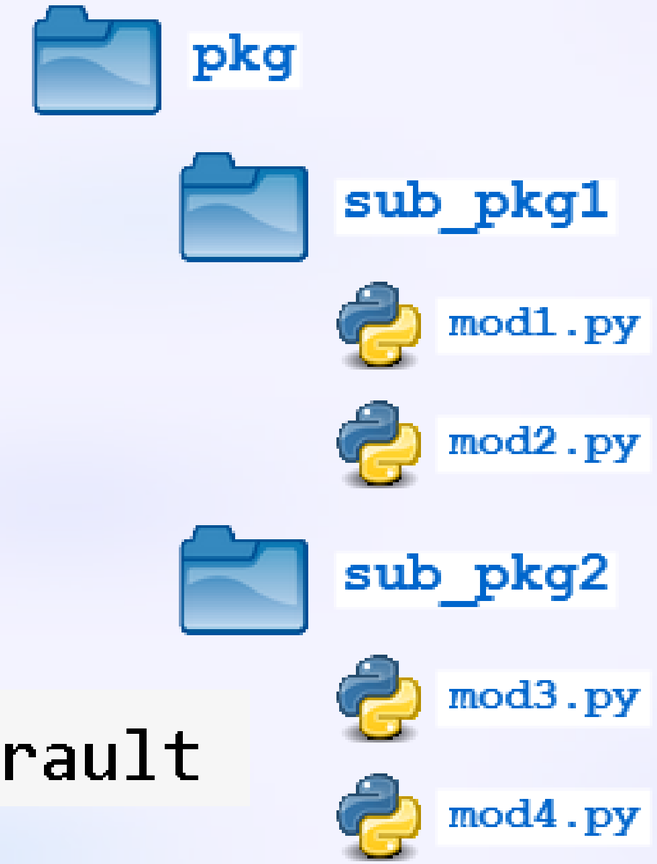
3 — Controlled Import

- Control what is imported from a package using the `__all__` list in `__init__.py`.
- __all__ can also be used inside a module to control what to import with *

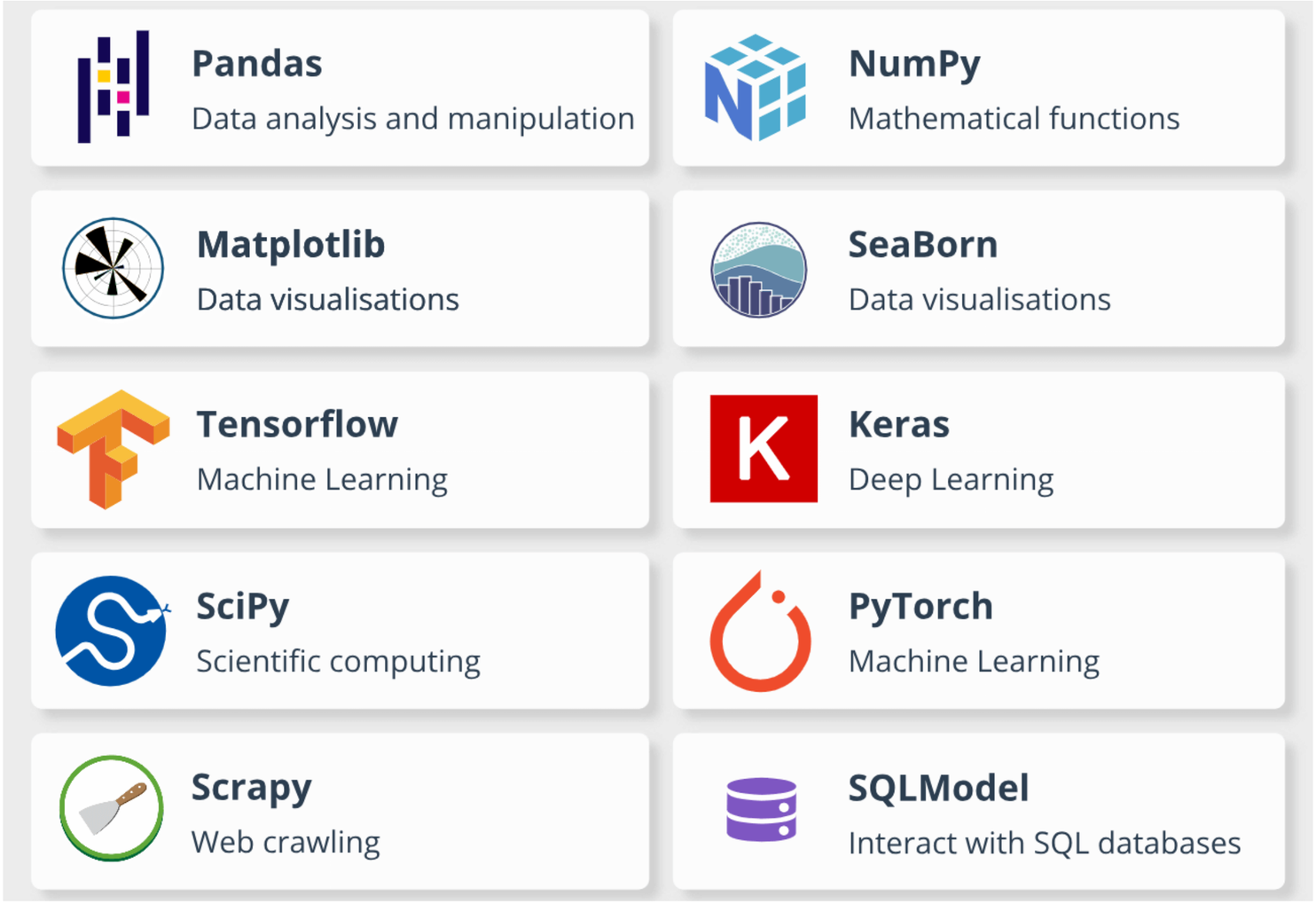
Subpackages Flexible Organization

Create nested subpackages within packages for better organization of modules.

```
import pkg.sub_pkg1.mod1

from pkg.sub_pkg1 import mod2

from pkg.sub_pkg2.mod3 import baz

from pkg.sub_pkg2.mod4 import qux as grault
```



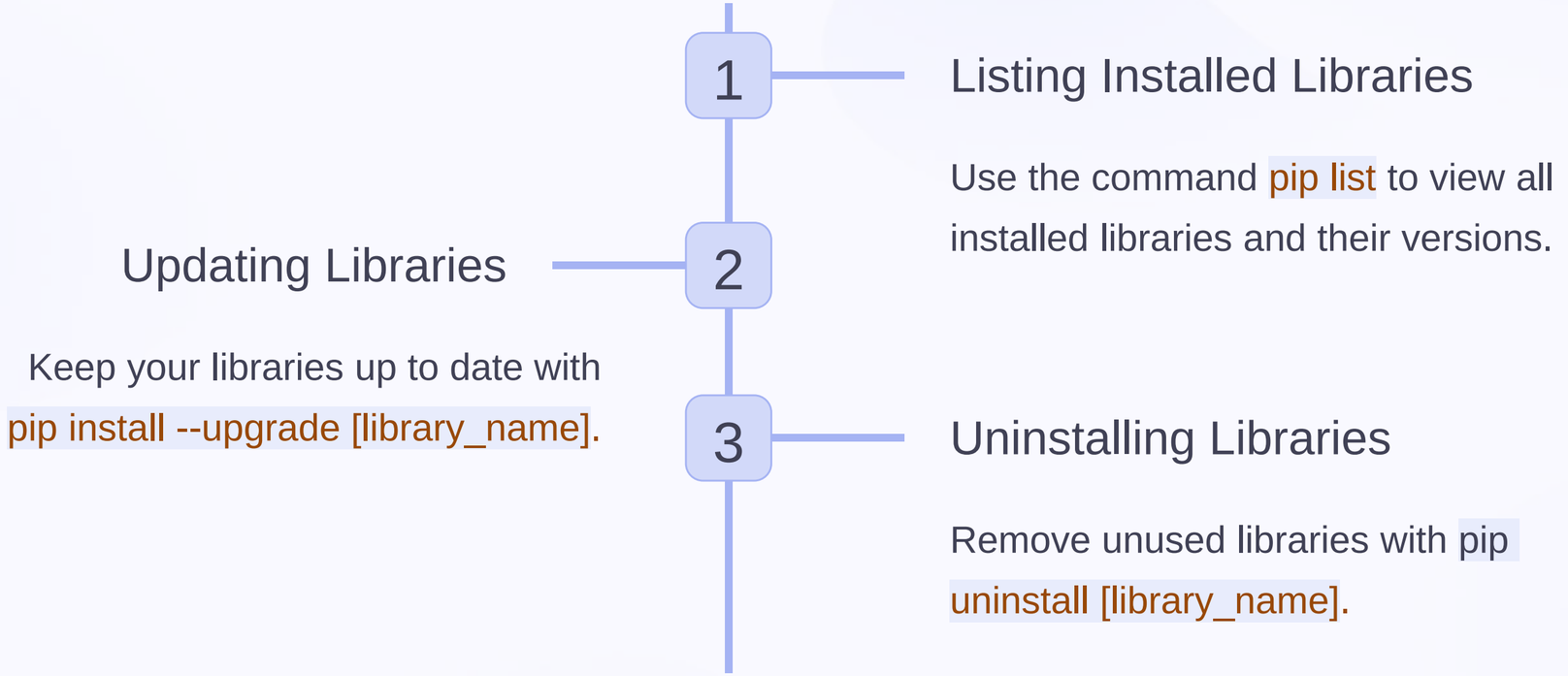
Working with External Python Packages and Libraries



Installing External Libraries

1. Use `pip install [library_name]` to install any library you want.
2. Ensure you have an active internet connection to download the library.
3. Once installed, import the library into your Python script using `import [library_name]`.

Managing External Libraries



Working with Imported Libraries

1 Importing a Library

Use the `import` statement to import the library into your script.

2 Using Library Functions

Access and use the functions provided by the library in your code.

3 Reading Documentation

Explore the library's documentation for detailed usage instructions and examples.

Conclusion

Python Module Development

1 A Modular Approach

Create reusable and organized code with Python modules.

2 Import and Access

Learn how to import modules and access their defined objects.

3 Executable Scripts

Create modules that can be executed as standalone scripts.

4 Package and Subpackage Organization

Organize your modules into packages and subpackages for better organization.

5 Initialization Control

Understand how to control the initialization of your packages.